

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS:

Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly. **Questions:**

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

Code of Conduct:

I (S.SHASHI VARDHAN REDDY/192311155) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

SHASHI VARDHAN

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers

Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

1: Online Symposium Portal

1. Identify the missing HTML5 semantic elements

The form is missing semantic HTML5 elements such as:

<header> – Page heading

<main> – Main content

<section> – Registration section <label>

– Labels for input fields <footer> –

Footer information

2. Standards-compliant HTML structure

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Online Symposium Registration</title>
```

```
</head>
```

```
<body>
```

```
<header>
```

```
  <h1>Online Symposium Registration</h1>
```

```
</header>
```

```
<main>
```

```
  <section>
```

```
    <form action="/register" method="POST">
```

```
      <label for="name">Name:</label>
```

```
      <input type="text" id="name" name="name" required>
```

```
      <label for="email">Email:</label>
```

```
      <input type="email" id="email" name="email" required>
```

```
      <button type="submit">Submit</button>
```

```
    </form>
```

```
  </section>
```

```
</main>
```

```
<footer>
```

```
<p>&copy; 2026 College Symposium</p>
</footer>

</body>
</html>
```

Online Symposium Registration

Name: Email:

© 2026 College Symposium

3. Interpret the HTTP request generated

When the user submits the form, the browser sends a **POST** request.

Example:

- POST /register HTTP/1.1
Host: www.college.edu
Content-Type: application/x-www-form-urlencoded
name=John&email=john@example.com •**POST** – Sends form data to the server.
•**/register** – Server endpoint.
•**Content-Type** – Indicates form data format.
•**Body** – Contains the submitted name and email.

4. Analyze the HTTP response cycle

1. User fills out the form and clicks **Submit**.
2. Browser sends an HTTP **POST** request.
3. Server receives and processes the data.
4. Server stores or validates the registration details.
5. Server returns an HTTP response (e.g., **200 OK** or **302 Redirect**).
6. Browser displays a success page or confirmation message.

Example Response:

HTTP/1.1 200 OK
Content-Type: text/html
Registration Successful!

5. Suggestions for usability and accessibility

- Use `<label>` for every input field.
- Add the required attribute for mandatory fields.
- Use meaningful button text such as **Register**.
- Add placeholder text if needed.

- Ensure keyboard accessibility.
- Use proper heading hierarchy (<h1>).
- Display clear error and success messages.
- Make the page responsive using the viewport meta tag.
- These improvements make the form **more user-friendly**, accessible, and HTML5 standards-compliant.

2: Cross-Browser Compatibility Issue

1. Identify improper nesting and missing closing tags

Problems in the HTML:

- Missing <!DOCTYPE html>.
- Missing <meta charset="UTF-8">.
- Missing closing </h1> tag.
- Missing closing </p> tag.
- Improper nesting of elements.
- Missing semantic HTML5 elements.

2. Analyze how different browsers interpret it

- Browsers automatically try to fix missing tags.
- Chrome, Firefox, and Edge may display the page correctly by auto-closing tags.
- Some older browsers may render the layout differently.
- This can lead to inconsistent formatting across browsers.

3. Identify missing semantic elements

The page should include:

<header> – Website header

<main> – Main content

<section> – About Us content <footer>

– Footer (optional)

4. Corrected W3C-compliant HTML structure

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Company</title>
```

```
</head>
```

```
<body>
```

```
<header>
```

```
  <h1>Welcome</h1>
```

```
</header>
```

```
<main>
```

```
<section>
  <p>About us</p>
</section>
</main>

<footer>
<p>&copy; 2026 Company</p>
</footer>

</body>
</html>
```

Welcome

About us

© 2026 Company

5. Techniques to ensure cross-browser compatibility

- Use valid HTML5 (<!DOCTYPE html>).
- Close all HTML tags properly.
- Use semantic HTML elements (<header>, <main>, <section>).
- Validate HTML using the W3C Validator.
- Test the website in multiple browsers (Chrome, Firefox, Edge, Safari).
- Use CSS resets or Normalize.css for consistent styling.
- Avoid browser-specific features unless necessary.
- Write responsive HTML and CSS.

3: Form Submission Failure

1. Identify issues in the form configuration

Problems:

- The button uses type="button" instead of type="submit".
- The **GET** method is not suitable for sending passwords.
- No <label> elements for the input fields.
- Missing required attributes.

2. Analyze why the HTTP request is not triggered

- A button with type="button" does **not** submit the form.
- Since the form is never submitted, the browser does not send an HTTP request to the server.
- Changing it to type="submit" triggers form submission.

3. Interpret the role of the GET method

- GET sends data through the URL as query parameters.

Example:

- /submit?username=John&password=12345
- Passwords become visible in the browser's address bar, browser history, and server logs.
- Therefore, GET is not secure for login forms.

4. Corrected implementation

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login</title>
</head>
<body>
```

```
<form action="/submit" method="POST">
  <label for="username">Username:</label>
```

Username: Password:

```
<input type="text" id="username" name="username" required>

<label for="password">Password:</label>
<input type="password" id="password" name="password" required>

<button type="submit">Login</button>
</form>

</body>
</html>
```

5. Suggestions for secure data transmission

- Use the **POST** method instead of GET.
- Use **HTTPS** to encrypt data during transmission.
- Never send passwords in the URL.
- Validate input on both the client and server.
- Store passwords using secure hashing (e.g., bcrypt).
- Use strong authentication measures such as session management or multi-factor authentication.

Result: The corrected form securely sends login data to the server using an HTTP **POST** request over **HTTPS**.